

SIMATS ENGINEERING

CO3_ASSESSMENT TOOL3_Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	Ganji Madhan Kumar
Name	192472328

COURSE OUTCOME COVERED

CO3: Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1.

Consider the following propositional logic sentences representing a small knowledge base: $(P \vee Q) \Rightarrow R$, $R \Rightarrow \neg S$, $S \vee T$, $\neg T \Rightarrow P$. (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of $\vee$ over $\wedge$). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α . Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of $KB \wedge \neg \alpha$, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) $Knows(John, x)$ and $Knows(John, Jane)$; (ii) $Knows(x, Elizabeth)$ and $Knows(y, x)$; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.

Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

I G.Madhan kumar(192472328)that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:G.Madhan

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	10
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	10

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification-failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	10
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	10
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	10

Q1. CNF Conversion and CNF Program

(a) Convert each sentence into CNF

Given:

- 1.
- 2.
- 3.
- 4.

Sentence 1:

Step 1: Eliminate implication

Therefore,

becomes

Step 2: Push negation inward

Using De Morgan's law:

Therefore:

Step 3: Distribute over

Hence:

CNF:

Clauses:

Sentence 2:

Step 1: Eliminate implication

There are no further transformations required.

CNF:

Clause:

Sentence 3:

This is already in CNF.

CNF:

Clause:

Sentence 4:

Step 1: Eliminate implication

Step 2: Push negation inward

CNF:

Clause:

Final CNF of the complete KB

(b) Python program for CNF conversion

The following program implements:

- implication elimination
- negation normalization
- conversion to CNF
- distribution of OR over AND

class Var:

```
def __init__(self, name):
    self.name = name
```

```
def __repr__(self):
    return self.name
```

class Not:

```
def __init__(self, expr):
    self.expr = expr
```

```
def __repr__(self):
    return f"~{self.expr}"
```

class And:

```
def __init__(self, left, right):
    self.left = left
    self.right = right
```

```
def __repr__(self):
    return f"({self.left} & {self.right})"
```

class Or:

```
def __init__(self, left, right):
    self.left = left
    self.right = right
```

```
def __repr__(self):
    return f"({self.left} | {self.right})"
```

class Imp:

```
def __init__(self, left, right):
```

```
self.left = left
self.right = right
```

```
def __repr__(self):
    return f'({self.left} -> {self.right})'
```

Step 1: Eliminate implications

```
def eliminate_implications(expr):
    if isinstance(expr, Var):
        return expr

    if isinstance(expr, Not):
        return Not(eliminate_implications(expr.expr))

    if isinstance(expr, And):
        return And(
            eliminate_implications(expr.left),
            eliminate_implications(expr.right)
        )

    if isinstance(expr, Or):
        return Or(
            eliminate_implications(expr.left),
            eliminate_implications(expr.right)
        )

    if isinstance(expr, Imp):
        left = eliminate_implications(expr.left)
        right = eliminate_implications(expr.right)

        return Or(Not(left), right)
```

Step 2: Convert to Negation Normal Form

```
def to_nnf(expr, neg=False):
    if isinstance(expr, Var):
        return Not(expr) if neg else expr

    if isinstance(expr, Not):
        return to_nnf(expr.expr, neg=True)
```

```
return to_nnf(expr.expr, not neg)
```

```
if isinstance(expr, And):
```

```
    if neg:
```

```
        return Or(
```

```
            to_nnf(expr.left, True),
```

```
            to_nnf(expr.right, True)
```

```
        )
```

```
    else:
```

```
        return And(
```

Output

Sentence 1:

Original : $((P \mid Q) \rightarrow R)$

CNF : $((\sim P \mid R) \& (\sim Q \mid R))$

Sentence 2:

Original : $(R \rightarrow \sim S)$

CNF : $(\sim R \mid \sim S)$

Sentence 3:

Original : $(S \mid T)$

CNF : $(S \mid T)$

Sentence 4:

Original : $(\sim T \rightarrow P)$

CNF : $(T \mid P)$

Q2. Propositional Resolution

We use a small Wumpus-World-style example.

Let:

- : Breeze at cell
- : Pit at cell
- : Pit at cell
- : Cell is safe

Knowledge base:

- 1.
- 2.
- 3.
- 4.

We want to prove:

(a) CNF of

Negate the query:

By De Morgan:

Therefore:

contains:

(b) Resolution steps

Resolution 1

Resolve:

with

The complementary literals are and .

Therefore:

Resolution 2

Resolve:

with:

Result:

Resolution 3

Resolve:

with:

Result:

The empty clause has been derived.

Therefore:

is unsatisfiable.

Hence:

(c) Python implementation of Propositional Resolution

```
def resolve(ci, cj):
```

```
    resolvents = []
```

```
    for literal in ci:
```

```
        if literal.startswith("~"):
```

```
            complement = literal[1:]
```

```
        else:
```

```
            complement = "~" + literal
```

```
    if complement in cj:
```

```
        new_clause = (ci - {literal}) | (cj - {complement})
```

```
        resolvents.append(frozenset(new_clause))
```


return resolvents

```
def pl_resolution(kb, query):
    clauses = set(frozenset(c) for c in kb)

    # Add negated query
    if isinstance(query, list):
        clauses.add(frozenset(
            "~" + literal if not literal.startswith("~")
            else literal[1:]
            for literal in query
        ))

    step = 1

    while True:
        new = set()
        clause_list = list(closures)

        for i in range(len(clause_list)):
            for j in range(i + 1, len(clause_list)):

                resolvents = resolve(
                    clause_list[i],
                    clause_list[j]
                )

                for resolvent in resolvents:
                    print(
                        f"Step {step}: "
                        f"{set(clause_list[i])} + "
                        f"{set(clause_list[j])} "
                        f"-> {set(resolvent)}"
                    )

                step += 1

                if len(resolvent) == 0:
                    return True
```

```

        new.add(resolvent)

    if new.issubset(clauses):
        return False

    clauses.update(new)

# KB
kb = [
    {"B11"},
    {"~B11", "P12", "P21"},
    {"~P12", "Safe11"},
    {"~P21", "Safe11"},
]

# Negated query is added separately
kb.append({"~P12"})
kb.append({"~P21"})

print("Resolution Result:")

result = pl_resolution(kb, [])

print("\nEntailment:", result)
Expected important output
Step 1: {'B11'} + {'~B11', 'P12', 'P21'} -> {'P12', 'P21'}

Step 2: {'P12', 'P21'} + {'~P12'} -> {'P21'}

Step 3: {'P21'} + {'~P21'} -> set()

Entailment: True

```

Q3. Unification Algorithm

The purpose of unification is to find substitutions that make two first-order expressions identical.

For example:

and
can be unified using:

(i) Knows(John, x) and Knows(John, Jane)

Compare arguments:

John = John

x = Jane

Therefore:

(ii) Knows(x, Elizabeth) and Knows(y, x)

Compare:

x = y

Elizabeth = x

From the second equation:

Since :

Therefore:

(iii) Example that fails

Consider:

and

The equations are:

and

This actually succeeds, so it is not a good occurs-check failure example.

A proper occurs-check example is:

and

We need:

But occurs inside .

Therefore the substitution would cause an infinite structure:

Hence:

Python implementation of UNIFY

```
def is_variable(x):
```

```
    return isinstance(x, str) and x.islower()
```

```
def occurs_check(var, term, substitution):
```

```
    if var == term:
```

```
        return True
```

```
if isinstance(term, tuple):
    return any(
        occurs_check(var, arg, substitution)
        for arg in term[1:]
    )
```

```
return False
```

```
def apply_substitution(term, substitution):
```

```
    if isinstance(term, str):
        while term in substitution:
            term = substitution[term]
        return term
```

```
    if isinstance(term, tuple):
```

```
        return (
            term[0],
            *[apply_substitution(arg, substitution)
              for arg in term[1:]]
        )
```

```
    return term
```

```
def unify(x, y, substitution=None):
```

```
    if substitution is None:
        substitution = {}
```

```
    x = apply_substitution(x, substitution)
```

```
    y = apply_substitution(y, substitution)
```

```
    if x == y:
```

```
        return substitution
```

```
    if is_variable(x):
```

```
        if occurs_check(x, y, substitution):
            return None
```

```
        substitution[x] = y
```

```
    return substitution
```

```
if is_variable(y):  
    if occurs_check(y, x, substitution):  
        return None
```

```
    substitution[y] = x  
    return substitution
```

```
if isinstance(x, tuple) and isinstance(y, tuple):
```

```
    if x[0] != y[0] or len(x) != len(y):  
        return None
```

```
    for a, b in zip(x[1:], y[1:]):  
        substitution = unify(a, b, substitution)
```

```
    if substitution is None:  
        return None
```

```
    return substitution
```

```
return None
```

```
# Test 1
```

```
a = ("Knows", "John", "x")
```

```
b = ("Knows", "John", "Jane")
```

```
print("Test 1:", unify(a, b))
```

```
# Test 2
```

```
a = ("Knows", "x", "Elizabeth")
```

```
b = ("Knows", "y", "x")
```

```
print("Test 2:", unify(a, b))
```

```
# Test 3
```

```
a = ("P", "x")  
b = ("P", ("f", "x"))
```

```
print("Test 3:", unify(a, b))
```

Output

Test 1: {'x': 'Jane'}

Test 2: {'x': 'Elizabeth', 'y': 'Elizabeth'}

Test 3: None

Q4. First-Order Resolution

We use an animal classification domain.

Knowledge Base

Let the following sentences be given.

F1

All cats are animals.

F2

All animals are living things.

F3

All mammals are animals.

F4

All cats are mammals.

F5

Tom is a cat.

Goal

Prove:

(a) Convert KB and negated goal to CNF

Sentence 1

Eliminate implication:

Clause:

Sentence 2

CNF:

Clause:

Sentence 3

CNF:

Clause:

Sentence 4

CNF:

Clause:

Sentence 5

Already CNF:

Negated goal

Goal:

Negate:

Therefore:

(b) First-Order Resolution

We now use resolution with unification.

Step 1

Take:

and

Unifier:

Resolving:

Therefore:

Step 2

Take:

and:

Unifier:

Resolvent:

Step 3

Take:

and:

Unifier:

Resolvent:

Step 4

Take:

and negated goal:

They contain complementary literals.

Therefore:

The empty clause is obtained.

Thus:

Resolution proof trace

C4: $\neg \text{Cat}(x) \vee \text{Mammal}(x)$

C5: $\text{Cat}(\text{Tom})$

$\theta_1 = \{x/\text{Tom}\}$

C7: $\text{Mammal}(\text{Tom})$

C3: $\neg \text{Mammal}(x) \vee \text{Animal}(x)$

C7: $\text{Mammal}(\text{Tom})$

$\theta_2 = \{x/\text{Tom}\}$

C8: $\text{Animal}(\text{Tom})$

C2: $\neg \text{Animal}(x) \vee \text{Living}(x)$

C8: $\text{Animal}(\text{Tom})$

$\theta_3 = \{x/\text{Tom}\}$

C9: $\text{Living}(\text{Tom})$

C9: $\text{Living}(\text{Tom})$

C6: $\neg \text{Living}(\text{Tom})$

□

First-Order Resolution Python Program

```
def substitute(term, substitution):
```

```
    if isinstance(term, str):
```

```
        while term in substitution:
```

```
            term = substitution[term]
```

```
    return term
```

```
    if isinstance(term, tuple):
```

```
        return (
```

```
            term[0],
```

```
            *[substitute(x, substitution)
```



```
        for x in term[1:]
    )
```

```
return term
```

```
def is_variable(term):
    return isinstance(term, str) and term.islower()
```

```
def occurs(var, term, substitution):
    term = substitute(term, substitution)
```

```
    if var == term:
        return True
```

```
    if isinstance(term, tuple):
        return any(
            occurs(var, arg, substitution)
            for arg in term[1:]
        )
```

```
    return False
```

```
def unify_term(x, y, substitution=None):
    if substitution is None:
        substitution = {}
```

```
    x = substitute(x, substitution)
    y = substitute(y, substitution)
```

```
    if x == y:
        return substitution
```

```
    if is_variable(x):
        if occurs(x, y, substitution):
            return None
        substitution[x] = y
        return substitution
```

```

if is_variable(y):
    if occurs(y, x, substitution):
        return None
    substitution[y] = x
    return substitution

if isinstance(x, tuple) and isinstance(y, tuple):
    if x[0] != y[0] or len(x) != len(y):
        return None

    for a, b in zip(x[1:], y[1:]):
        substitution = unify_term(a, b, substitution)

    if substitution is None:
        return None

    return substitution

return None

```

```

# Example FOL resolution
clauses = [
    {"~Cat(x)", "Mammal(x)"},
    {"~Mammal(x)", "Animal(x)"},
    {"~Animal(x)", "Living(x)"},
    {"Cat(Tom)"},
    {"~Living(Tom)"}
]

```

Q5. Forward and Backward Chaining

We construct a simple smart agricultural system.

Knowledge Base

Facts

1. Rain
2. WetSoil
3. HealthyCrop

Rules

1. If it rains, then the soil becomes wet.
2. If the soil is wet, then the soil has moisture.
3. If there is moisture, then the crop can grow.
4. If the crop grows and is healthy, then yield is good.
5. If there is good yield, then the farm is productive.

Query

(a) Forward Chaining

Forward chaining starts from the known facts and repeatedly applies rules.

Initial facts

Rain

WetSoil

HealthyCrop

Iteration 1

Rule 1:

Rain $\rightarrow$ WetGround

Therefore:

WetGround

Facts now:

Rain

WetSoil

HealthyCrop

WetGround

Iteration 2

Rule 2:

WetGround $\rightarrow$ Moisture

Therefore:

Moisture

Facts:

Rain

WetSoil

HealthyCrop

WetGround

Moisture

Iteration 3

Rule 3:

Moisture $\rightarrow$ CropGrowth

Therefore:
CropGrowth

Iteration 4

Rule 4:

CropGrowth AND HealthyCrop $\rightarrow$ GoodYield

Both conditions are true.

Therefore:

GoodYield

Iteration 5

Rule 5:

GoodYield $\rightarrow$ ProductiveFarm

Therefore:

ProductiveFarm

The query is derived.

Forward Chaining Python Program

```
facts = {  
    "Rain",  
    "WetSoil",  
    "HealthyCrop"  
}
```

```
rules = [  
    ({"Rain"}, "WetGround"),  
    ({"WetGround"}, "Moisture"),  
    ({"Moisture"}, "CropGrowth"),  
    ({"CropGrowth", "HealthyCrop"}, "GoodYield"),  
    ({"GoodYield"}, "ProductiveFarm")  
]
```

```
def forward_chaining(facts, rules, query):
```

```
    facts = set(facts)
```

```
    iteration = 1
```

```
    while True:
```

```
new_facts = set()
```

```
print(f"\nIteration {iteration}")
```

```
for conditions, conclusion in rules:
```

```
    if conditions.issubset(facts):
```

```
        if conclusion not in facts:
```

```
            new_facts.add(conclusion)
```

```
        print(
```

```
            f"{conditions} -> {conclusion}"
```

```
        )
```

```
    if not new_facts:
```

```
        break
```

```
facts.update(new_facts)
```

```
print("New facts:", new_facts)
```

```
if query in facts:
```

```
    return True
```

```
iteration += 1
```

```
return query in facts
```

```
result = forward_chaining(
```

```
    facts,
```

```
    rules,
```

```
    "ProductiveFarm"
```

```
)
```

```
print("\nQuery:", "ProductiveFarm")
```

```
print("Entailed:", result)
```

Output

Iteration 1

{'Rain'} -> WetGround

New facts: {'WetGround'}

Iteration 2

{'WetGround'} -> Moisture

New facts: {'Moisture'}

Iteration 3

{'Moisture'} -> CropGrowth

New facts: {'CropGrowth'}

Iteration 4

{'CropGrowth', 'HealthyCrop'} -> GoodYield

New facts: {'GoodYield'}

Iteration 5

{'GoodYield'} -> ProductiveFarm

New facts: {'ProductiveFarm'}

Query: ProductiveFarm

Entailed: True

(b) Backward Chaining

Backward chaining starts with the query and works backward toward known facts.

Our query is:

ProductiveFarm

Goal 1

ProductiveFarm

Find a rule that produces it:

GoodYield → ProductiveFarm

New subgoal:

GoodYield

Goal 2

Find a rule that produces GoodYield:

CropGrowth AND HealthyCrop → GoodYield

Therefore we need:

CropGrowth

HealthyCrop

Goal 3

For CropGrowth:

Moisture $\rightarrow$ CropGrowth

New subgoal:

Moisture

Goal 4

For Moisture:

WetGround $\rightarrow$ Moisture

New subgoal:

WetGround

Goal 5

For WetGround:

Rain $\rightarrow$ WetGround

New subgoal:

Rain

Rain is a known fact.

Also:

HealthyCrop

is a known fact.

Therefore:

Rain

↓

WetGround

↓

Moisture

↓

CropGrowth

↓

CropGrowth + HealthyCrop

↓

GoodYield

↓

ProductiveFarm

Hence:

Backward Chaining Python Program

```
facts = {  
    "Rain",  
    "WetSoil",  
    "HealthyCrop"  
}
```

```
rules = [  
    ("Rain", "WetGround"),  
    ("WetGround", "Moisture"),  
    ("Moisture", "CropGrowth"),  
    ("CropGrowth", "HealthyCrop", "GoodYield"),  
    ("GoodYield", "ProductiveFarm")  
]
```

```
def backward_chaining(goal, facts, rules, visited=None):
```

```
    if visited is None:
```

```
        visited = set()
```

```
    print("Trying to prove:", goal)
```

```
    # Base case
```

```
    if goal in facts:
```

```
        print("Fact found:", goal)
```

```
        return True
```

```
    if goal in visited:
```

```
        return False
```

```
    visited.add(goal)
```

```
    # Find rules that conclude the goal
```

```
    for conditions, conclusion in rules:
```

```
        if conclusion == goal:
```

```
            print(  
                f"Rule found: "
```

```
                f"{ ' AND '.join(conditions)} -> {goal}"
```

```
            )
```

```
            all_true = True
```

```
            for condition in conditions:
```



```
        if not backward_chaining(
            condition,
            facts,
            rules,
            visited
        ):
            all_true = False
            break
```

```
    if all_true:
        return True
```

```
    return False
```

```
result = backward_chaining(
    "ProductiveFarm",
    facts,
    rules
)
```

```
print("\nQuery:", "ProductiveFarm")
print("Entailed:", result)
Backward chaining trace
Trying to prove: ProductiveFarm
```

```
Rule found:
GoodYield -> ProductiveFarm
```

```
Trying to prove: GoodYield
```

```
Rule found:
CropGrowth AND HealthyCrop -> GoodYield
```

```
Trying to prove: CropGrowth
```

```
Rule found:
Moisture -> CropGrowth
```

```
Trying to prove: Moisture
```

Rule found:
WetGround -> Moisture

Trying to prove: WetGround

Rule found:
Rain -> WetGround

Trying to prove: Rain

Fact found: Rain

Trying to prove: HealthyCrop

Fact found: HealthyCrop

Query: ProductiveFarm
Entailed: True

(c) Forward vs Backward Chaining

Feature	Forward Chaining	Backward Chaining
Starting point	Known facts	Query/goal
Direction	Facts → conclusion	Goal → required facts
Approach	Data-driven	Goal-driven
Main operation	Apply rules whose conditions are satisfied	Find rules that can prove the goal
Stops when	Query is derived or no new facts exist	Goal is proved or all possible rules fail
Advantage	Good when many facts are available	Good when there is a specific query
Disadvantage	May derive unnecessary facts	Can repeatedly explore rules
Our example	Derives Rain → ProductiveFarm	Starts ProductiveFarm → GoodYield → ... → Rain

Query result	Entailed	Entailed
--------------	----------	----------